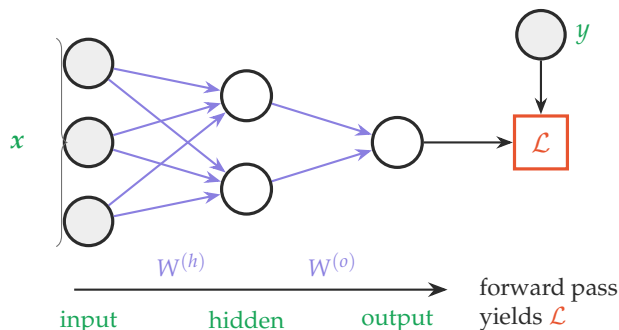


Backpropagation (1)

DS 4440 · LECTURE 5

1. Last time

Computation graphs comprise **layers** to form **networks**.



Today: how to adjust **parameters** in computation graphs to minimize $\mathcal{L}$ — **backprop**. This extends SGD (L1; HW2).

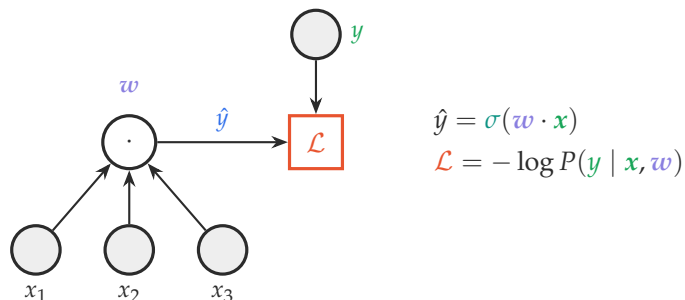
2. The recipe

- Build a **graph** (the model) with parameters w
- Define a (differentiable) loss $\mathcal{L}$
- Use $\nabla_w \mathcal{L}$ to find $\hat{w}$ via SGD

Problem. Finding ∇_w is painful to do manually for big models.

Solution. Use the computation graph to auto-differentiate via **backprop**.

3. Warm-up: logistic regression by hand



Here $x, w \in \mathbb{R}^{(d \times 1)}$ and $z = w \cdot x$ is a scalar, so $\nabla_w \mathcal{L}$ is $\langle d \times 1 \rangle$ — the shape of w . Take binary cross entropy as the loss:

$$\mathcal{L} = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})].$$

Step 1: $\partial \mathcal{L} / \partial \hat{y}$

Both y and $(1 - y)$ are constants with respect to $\hat{y}$, so we differentiate two terms of the form (constant) $\cdot \log(\text{something})$. The first is immediate:

$$\frac{\partial}{\partial \hat{y}} [y \log \hat{y}] = \frac{y}{\hat{y}}.$$

The second needs the chain rule, since the argument of the log is $1 - \hat{y}$ rather than $\hat{y}$. With $u = 1 - \hat{y}$:

$$\frac{\partial}{\partial \hat{y}} [(1 - y) \log u] = (1 - y) \cdot \frac{1}{u} \cdot \underbrace{\frac{\partial u}{\partial \hat{y}}}_{-1} = -\frac{1 - y}{1 - \hat{y}}.$$

That -1 is where the minus sign inside the bracket comes from; it is the derivative of $1 - \hat{y}$, not the leading minus of the loss. Putting the two together:

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = -\left[\frac{y}{\hat{y}} - \frac{1 - y}{1 - \hat{y}}\right].$$

Since y is 0 or 1, one term always drops out: for $y = 1$ the derivative is $-1/\hat{y}$, and for $y = 0$ it is $+1/(1 - \hat{y})$. Both blow up as the prediction approaches the wrong extreme.

Step 2: the rest of the chain

The remaining two local derivatives are

$$\frac{\partial \hat{y}}{\partial z} = \hat{y}(1 - \hat{y}), \quad \frac{\partial z}{\partial w} = x.$$

Multiplying the first two factors, the $\hat{y}(1 - \hat{y})$ cancels both denominators:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial z} &= -\left[\frac{y}{\hat{y}} - \frac{1 - y}{1 - \hat{y}}\right] \hat{y}(1 - \hat{y}) \\ &= -[y(1 - \hat{y}) - (1 - y)\hat{y}] = -[y - \hat{y}] = \hat{y} - y, \end{aligned}$$

where the $y\hat{y}$ terms cancel in the middle step. Multiplying by $\partial z / \partial w = x$ gives, for one instance (x, y) ,

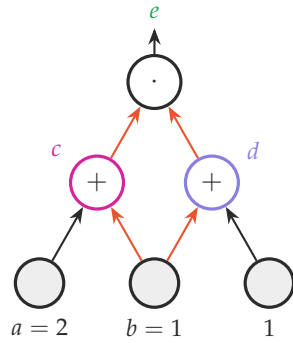
$$\nabla_w \mathcal{L} = (\sigma(w \cdot x) - y) x$$

Sigmoid and BCE are paired for this reason: the σ' factor cancels, and the gradient is (prediction - target) times the input. Note the sign. The gradient of the *log-likelihood* is $(y - \sigma(w \cdot x))x$, which we would ascend; here we descend $\mathcal{L} = -\log P$.

4. Why not just sum over paths?

Recall from Lecture 4 that a gradient is (incoming gradient) $\times$ (local derivative), and that when a value feeds more than one node, the contributions add. Take

$$e = (\underbrace{a+b}_c)(\underbrace{b+1}_d), \quad a = 2, b = 1, \quad \frac{\partial e}{\partial b} = ?$$



two paths from b to e :

$$\frac{\partial e}{\partial b} = \underbrace{\frac{\partial e}{\partial c} \frac{\partial c}{\partial b}}_{(b+1)} + \underbrace{\frac{\partial e}{\partial d} \frac{\partial d}{\partial b}}_{(a+b)} = 2 + 3 = 5$$

Summing over paths is correct, and for a graph this size it is fine. It does not scale. In a network with L layers of width d , a weight in the first layer reaches the loss along roughly d^{L-1} distinct paths:

$L = 10$ layers of width $d = 100$: $\approx 100^9 = 10^{18}$ paths, versus $\approx 10^5$ edges in the whole graph.

These paths overlap: any two that meet at a node share everything above it. Backprop computes one quantity *per node* rather than one term per path, and reuses it for everything below that node. The cost is then linear in the number of edges rather than exponential in depth.

5. δ : the node error

We give that reused quantity a name. Write $\mathbf{z}^{(l)}$ for the pre-activation of layer l and $\mathbf{h}^{(l)} = \sigma(\mathbf{z}^{(l)})$ for its output. Throughout this lecture a single example is a *column* vector and $\mathbf{W}^{(l)}$ is (units out) $\times$ (units in), so with d_l units in layer l :

$$\mathbf{h}^{(l)}, \mathbf{z}^{(l)} \in \mathbb{R}^{\langle d_l \times 1 \rangle}, \quad \mathbf{W}^{(l)} \in \mathbb{R}^{\langle d_l \times d_{l-1} \rangle}, \quad \mathbf{b}^{(l)} \in \mathbb{R}^{\langle d_l \times 1 \rangle}.$$

Define

$$\delta^{(l)} \stackrel{\text{def}}{=} \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}}$$

the *node error* at layer l : how much the loss cares about that layer's pre-activations.

$$\delta^{(l)} \in \mathbb{R}^{\langle d_l \times 1 \rangle}, \text{ the shape of } \mathbf{z}^{(l)}.$$

Keep this distinct from the *local derivative* of Lecture 4. In the rule (incoming) $\times$ (local), a node's local derivative is a property of that one operation; δ is a gradient that has travelled down to the node. When we use $\delta^{(l+1)}$ to compute $\delta^{(l)}$, it is playing the incoming role.

Everything we need follows from δ . Once we have it at a layer, the parameter gradients there are immediate:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \underbrace{\underbrace{\delta^{(l)}}_{\langle d_l \times 1 \rangle} \underbrace{(\mathbf{h}^{(l-1)})^\top}_{\langle 1 \times d_{l-1} \rangle}}_{\langle d_l \times d_{l-1} \rangle \checkmark}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} = \underbrace{\delta^{(l)}}_{\langle d_l \times 1 \rangle}.$$

And δ at one layer determines δ at the layer below:

$$\delta^{(l)} = \underbrace{\left(\underbrace{W^{(l+1)\top}}_{\langle d_l \times d_{l+1} \rangle} \underbrace{\delta^{(l+1)}}_{\langle d_{l+1} \times 1 \rangle} \right)}_{\langle d_l \times 1 \rangle} \odot \underbrace{\sigma'(z^{(l)})}_{\langle d_l \times 1 \rangle}$$

Reading right to left: take the error from the layer above, send it back through the weights that carried the signal forward ($W^{(l+1)\top}$), then scale by how responsive this layer's activation was (σ'). Start with δ at the output and apply this repeatedly.

Where the two factors come from

This is (incoming) $\times$ (local derivative) applied twice, because going from $z^{(l)}$ up to $z^{(l+1)}$ takes two hops:

$$\begin{array}{ccccccc} z^{(l)} & \xrightarrow{\sigma} & h^{(l)} & \xrightarrow{W^{(l+1)}} & z^{(l+1)} & \longrightarrow \dots \longrightarrow & \mathcal{L} \\ \delta^{(l)} & & & & \delta^{(l+1)} & & \end{array}$$

Hop 1 (through the weights). $z^{(l+1)} = W^{(l+1)}h^{(l)} + b^{(l+1)}$, so the local derivative of unit j above with respect to unit i here is $\partial z_j^{(l+1)} / \partial h_i^{(l)} = W^{(l+1)}_{ji}$. Unit i feeds *every* unit above it, so by the fan-out rule its contributions add:

$$\frac{\partial \mathcal{L}}{\partial h_i^{(l)}} = \sum_j \underbrace{\delta_j^{(l+1)}}_{\text{incoming}} \underbrace{W^{(l+1)}_{ji}}_{\text{local}} \iff \frac{\partial \mathcal{L}}{\partial h^{(l)}} = W^{(l+1)\top} \delta^{(l+1)}.$$

The sum runs over j , the *first* index of $W^{(l+1)}$, which is why the matrix form carries the transpose.

Hop 2 (through the activation). $h_i^{(l)} = \sigma(z_i^{(l)})$, a function of $z_i^{(l)}$ alone, so the local derivative is $\sigma'(z_i^{(l)})$ and there is no sum:

$$\delta_i^{(l)} = \frac{\partial \mathcal{L}}{\partial z_i^{(l)}} = \underbrace{\frac{\partial \mathcal{L}}{\partial h_i^{(l)}}}_{\text{incoming}} \underbrace{\sigma'(z_i^{(l)})}_{\text{local}}.$$

Because each output depends only on the matching input, this second step is elementwise — the diagonal Jacobian from Lecture 3, which Lecture 4 showed becomes a Hadamard product. Composing the two hops:

$$\delta_i^{(l)} = \sigma'(z_i^{(l)}) \sum_j W^{(l+1)}_{ji} \delta_j^{(l+1)} \iff \delta^{(l)} = (W^{(l+1)\top} \delta^{(l+1)}) \odot \sigma'(z^{(l)}),$$

which is the boxed recursion.

One indexing note: the local derivatives in the δ recursion involve this layer's own output $h^{(l)}$, not $h^{(l-1)}$. The previous layer's output shows up in the *weight* gradient, $\partial \mathcal{L} / \partial W^{(l)} = \delta^{(l)} (h^{(l-1)})^\top$, because $h^{(l-1)}$ is what $W^{(l)}$ multiplied on the way forward. Two different local derivatives of the same node: one with respect to its input, one with respect to its weights.

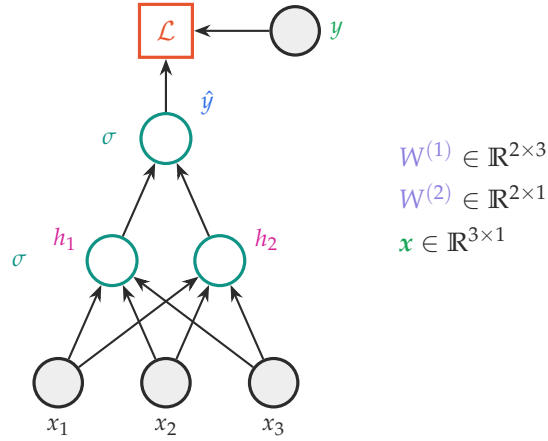
Two conventions, and why the formula looks reversed

Lecture 4 wrote this same gradient as $x^\top (\partial \mathcal{L} / \partial h)$, with the transpose on the *other* factor. Both are right; the two lectures lay out the data differently.

	Lecture 4 batch as <i>rows</i>	Lecture 5 (here) one example as a <i>column</i>
layout of W	$(\text{in}) \times (\text{out})$	$(\text{out}) \times (\text{in})$
input	$\mathbf{x} \in \langle N \times d \rangle$	$\mathbf{x} \in \langle d \times 1 \rangle$
weights	$W \in \langle d \times m \rangle$	$W \in \langle m \times d \rangle$
forward	$\mathbf{h} = \mathbf{x}W \in \langle N \times m \rangle$	$\mathbf{h} = W\mathbf{x} \in \langle m \times 1 \rangle$
gradient	$\frac{\partial \mathcal{L}}{\partial W} = \mathbf{x}^\top \frac{\partial \mathcal{L}}{\partial \mathbf{h}}$	$\frac{\partial \mathcal{L}}{\partial W} = \boldsymbol{\delta} \mathbf{h}^\top$
shape check	$\langle d \times N \rangle \langle N \times m \rangle = \langle d \times m \rangle \checkmark$	$\langle m \times 1 \rangle \langle 1 \times d \rangle = \langle m \times d \rangle \checkmark$

In both cases the gradient is the outer product of (what arrives from above) and (what came in from below); which factor carries the transpose is decided by whether W is $\langle d \times m \rangle$ or $\langle m \times d \rangle$. In index form there is no difference at all — the gradient for the wire from feature i to unit j is $x_i \delta_j$ either way, and the two matrices are transposes of each other. Lecture 4 used rows because it was carrying a batch of N examples, which is also how PyTorch stores data; this lecture uses columns because the backprop recursion reads more cleanly that way.

6. Worked example: a 3–2–1 network



$\nabla_{W^{(2)}} \mathcal{L}$ we have already seen — it is the single-layer case with $\mathbf{h}$ in place of $\mathbf{x}$. For the first layer, apply the recursion. The output layer's error is a scalar,

$$\delta_{\text{out}} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \sigma'(z_{\text{out}}),$$

and pushing it back through $W^{(2)}$ and the hidden activation gives one δ per hidden unit:

$$\delta_i = \delta_{\text{out}} W^{(2)}_i \sigma'(z_i), \quad i = 1, 2, \quad \text{so } \boldsymbol{\delta} \in \mathbb{R}^{(2 \times 1)}.$$

Then the weight gradient is the outer product $\boldsymbol{\delta} \mathbf{x}^\top$:

$$\nabla_{W^{(1)}} \mathcal{L} = \begin{bmatrix} \delta_1 \\ \delta_2 \end{bmatrix} \begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix} = \begin{bmatrix} \delta_1 x_1 & \delta_1 x_2 & \delta_1 x_3 \\ \delta_2 x_1 & \delta_2 x_2 & \delta_2 x_3 \end{bmatrix} = \begin{bmatrix} \delta_1 \vec{x} \\ \delta_2 \vec{x} \end{bmatrix},$$

which is $\langle 2 \times 1 \rangle \langle 1 \times 3 \rangle = \langle 2 \times 3 \rangle$ — the shape of $W^{(1)}$. Every entry is (node error at the unit this wire feeds) $\times$ (the feature this wire carries): activity times responsibility, one entry per wire.

Reading one row

Row i of $W^{(1)}$ holds the wires into hidden unit i , and they reach the loss only through z_i . So δ_i scales the whole row, and x decides how that is shared out: a feature that was large moves its wire a lot, and a feature that was zero does not move its wire at all. And since $\delta_i = \delta_{\text{out}} W^{(2)}_i \sigma'(z_i)$, a saturated unit has $\sigma'(z_i) \approx 0$, so $\delta_i \approx 0$ and the row does not move, however wrong the prediction was.

Consistency check against Lecture 4

For the two-layer regression model $\hat{y} = w_2(w_1x + b_1) + b_2$ with squared error and no activation, the recursion gives

$$\frac{\partial \mathcal{L}}{\partial h} = 2(\hat{y} - y)w_2, \quad \frac{\partial \mathcal{L}}{\partial w_1} = x \cdot 2(\hat{y} - y)w_2, \quad \frac{\partial \mathcal{L}}{\partial b_1} = 2(\hat{y} - y)w_2.$$

With the numbers from Lecture 4 ($x = 3$, $w_1 = 5$, $b_1 = 2$, $w_2 = 2$, $b_2 = 3$, $y = 40$, so $\hat{y} = 37$): -12 , -36 , and -12 — exactly the values in that backward sweep.

7. Taking the step

With a learning rate α , the update is

$$w \leftarrow w - \alpha \frac{\partial \mathcal{L}}{\partial w}, \quad \text{e.g.} \quad w_2 \leftarrow 2 - 0.01(-102) = 3.02.$$

Note the sign: the gradient was negative, so the loss decreases as w_2 grows, and the update moves it up.

In code, the whole training step is five lines:

```
optimizer.zero_grad()
y_hat = model(x)
loss = criterion(y_hat, y)
loss.backward()
optimizer.step()
```

`zero_grad()` is there because of the fan-out rule: gradients *accumulate* into `.grad`. That is what we want within a single backward pass, and not what we want across steps.

(See the Colab notebook. More next time.)

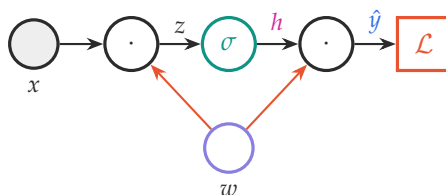
8. Some additional worked examples

Each of these breaks an assumption that the plain MLP quietly satisfies.

1. A shared weight

Use the *same* parameter in both layers: with $z = wx$, $h = \sigma(z)$ and $\hat{y} = wh$,

$$\hat{y} = w \sigma(wx).$$



w feeds two nodes, so the fan-out rule applies — this time to a *parameter*. Each use contributes (node error there) $\times$ (local derivative there):

$$\frac{\partial \mathcal{L}}{\partial w} = \underbrace{\delta_{\hat{y}} h}_{\text{outer use}} + \underbrace{\delta_z x}_{\text{inner use}}, \quad \delta_{\hat{y}} = \frac{\partial \mathcal{L}}{\partial \hat{y}}, \quad \delta_z = \delta_{\hat{y}} w \sigma'(z).$$

The expression for δ_z is the two-hop recursion again, in one dimension. To get from z to the loss we pass through σ , with local derivative $\sigma'(z)$, and then through the outer multiply $\hat{y} = w h$, with local derivative $\partial \hat{y} / \partial h = w$:

$$\delta_z = \underbrace{\delta_{\hat{y}}}_{\text{incoming}} \cdot \underbrace{w}_{\text{through } \times w} \cdot \underbrace{\sigma'(z)}_{\text{through } \sigma},$$

which is $(W^{(l+1)\top} \delta^{(l+1)}) \odot \sigma'$ with a scalar in place of the matrix.

Note that we treated w as a fixed multiplier while computing δ_z , even though w is what we are differentiating. That is allowed: for a variable used more than once, differentiate with respect to each occurrence separately, holding the others fixed, then add. Labelling the two uses w_a (inner) and w_b (outer), so that $\hat{y} = w_b \sigma(w_a x)$,

$$\frac{\partial \mathcal{L}}{\partial w_a} = \delta_{\hat{y}} w_b \sigma'(z) x, \quad \frac{\partial \mathcal{L}}{\partial w_b} = \delta_{\hat{y}} h, \quad \frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial w_a} + \frac{\partial \mathcal{L}}{\partial w_b} \Big|_{w_a=w_b=w}.$$

Take $x = 1$, $w = 2$, ReLU for σ , and $y = 1$ with squared error. Forward: $z = 2$, $h = 2$, $\hat{y} = 4$. Backward: $\delta_{\hat{y}} = 2(4 - 1) = 6$, $\delta_z = 6 \cdot 2 \cdot 1 = 12$, so

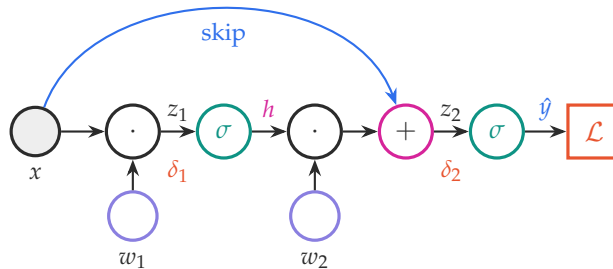
$$\frac{\partial \mathcal{L}}{\partial w} = 6 \cdot 2 + 12 \cdot 1 = 24.$$

Check it directly: with $x = 1$ and $w > 0$ the model is $\hat{y} = w^2$, so $\mathcal{L} = (w^2 - 1)^2$ and $d\mathcal{L}/dw = 4w(w^2 - 1) = 24$. Differentiating only the outer w would have given 12. This is what convolutions and RNNs do at scale, and it is why `.grad` accumulates.

2. A skip connection

Let the input reach the output twice, once through the layer and once around it:

$$h = \sigma(w_1 x), \quad \hat{y} = \sigma(w_2 h + x).$$



Walk the graph backwards one node at a time, naming the node error at each point:

point	node error	step taken
$\hat{y}$	$\delta_{\hat{y}} = \partial \mathcal{L} / \partial \hat{y}$	start
z_2	$\delta_2 = \delta_{\hat{y}} \sigma'(z_2)$	gate through σ
h	$\delta_2 w_2$	route through $\times w_2$
z_1	$\delta_1 = \delta_2 w_2 \sigma'(z_1)$	gate through σ

The $+$ node is what makes this interesting. Addition passes its incoming error to *both* inputs unchanged, so the error arriving at $w_2 h$ is δ_2 , and the error arriving along the skip edge is also δ_2 — the skip hands x the error from z_2 directly, without passing through w_2 or either σ . Now x enters the graph in two places, so its gradient is the sum of two terms, each one (node error where x enters) $\times$ (local derivative of that entry):

$$\frac{\partial \mathcal{L}}{\partial x} = \underbrace{\delta_1}_{\text{error at } z_1} \cdot \underbrace{w_1}_{\partial z_1 / \partial x} + \underbrace{\delta_2}_{\text{error at } z_2} \cdot \underbrace{1}_{\substack{\partial z_2 / \partial x \\ \text{via the skip}}} = \delta_2 w_2 \sigma'(z_1) w_1 + \delta_2.$$

Written that way the contrast is clear: the first term is the error from z_2 after it has been routed and gated twice on the way down, and the second is that same error arriving intact.

Take $w_1 = -1$, $x = 1$ with ReLU: then $z_1 = -1$, $h = 0$, and $\sigma'(z_1) = 0$, so $\delta_1 = 0$ and the deep path delivers nothing — but $\partial \mathcal{L} / \partial x = \delta_2$ regardless. In general, for $h^{(l+1)} = h^{(l)} + F(h^{(l)})$ the recursion picks up an identity term, $\delta^{(l)} = \delta^{(l+1)} (I + \partial F / \partial h^{(l)})$, which is why residual networks train at depth.

3. A multiplicative interaction

$$y = (w_1 x_1)(w_2 x_2),$$

$$\frac{\partial y}{\partial w_1} = x_1 (w_2 x_2).$$

Each parameter's gradient is scaled by the *other* branch: if $w_2 x_2 = 0$ then w_1 gets no gradient, however wrong the output was. Gating layers and attention rely on this coupling.

4. A max

$$y = \max(a, b),$$

$$\frac{\partial y}{\partial a} = \begin{cases} 1 & a > b \\ 0 & \text{otherwise} \end{cases}$$

Backward, a max is a router: the incoming gradient goes entirely to whichever input won, and the other gets zero. Max-pooling works this way, and with $b = 0$ it is ReLU. At $a = b$ we pick a side by convention.